МОСКОВСКИЙ УНИВЕРСИТЕТ ИМЕНИ С.Ю. ВИТТЕ

Руководитель практики
от профильного структурного
подразделения Университета*
_____/……../
«__»_____ 2026 г

Руководитель практики
от Университета
_____/ Д.П. Коротков /
03.04.2026 г.

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Студент Соколов Никита Евгеньевич
Направление подготовки: 09.03.03 Прикладная информатика
Направленность (профиль) Искусственный интеллект и анализ данных
Вид практики производственная практика
Тип практики Преддипломная практика
Срок прохождения практики 2026-04-13 – 2026-05-10
Место проведения практики (профильное подразделение Университета) кафедра информационных систем

С требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего трудового распорядка на предприятии ознакомлен.

13.04.2026 г. _____/ Соколов Никита Евгеньевич /
Подпись *ФИО обучающегося*

** для практики на базе Университета не вносится*

Период практики	Содержание работ	Планируемые результаты практики (формируемые компетенции)	Отметка о выполнении *
	Виды проделанной работы, результаты которой необходимо отразить в отчете:		
1 неделя	– оформить документы на практику, ознакомление с требованиями по прохождению практики и правилами оформления документов по практике. – Получить у руководителя практики от Университета и ознакомиться с индивидуальным заданием прохождения преддипломной практики.	ПК-1, ПК-7, ПК-9, ПК-10	вып.
	– ознакомиться со структурой Университета, основными учредительными документами, внутренней организационно-распорядительной документацией, нормативно-правовыми документами		

Период практики	Содержание работ	Планируемые результаты практики (формируемые компетенции)	Отметка о выполнении *
	и внутренними регламентами Университета в области ИТ-инфраструктуры на основе данных раздела официального сайта «Сведения об образовательной организации» и корпоративного портала – выбрать **один и/или несколько бизнес-процессов** согласно тематике ВКР, для которых необходимо разработать интеллектуальную систему – выбрать **подразделение**, ответственное за выбранный ранее бизнес-процесс – провести анализ требований пользователей к разрабатываемой информационной системе с элементами ИИ – выполнить постановку задачи для формализации процесса выбора подходящей модели обучения – **определить набор данных** извлечения, анализа и обработки информации – **выбрать модели** и инструментальные средства интеллектуального анализа больших данных – провести анализ вычислительных ресурсов и инфраструктуры для интеллектуального анализа данных – **выбрать DLM для решения поставленной задачи** – выбрать вариант интерфейсной части (front) информационной системы. – составить техническое задание на разработку / модификацию / проектирование / обучение / развертывание составляющих компонентов корпоративной информационной системы. – сформировать материалы для аналитической частей выпускной квалификационной работы		
2-3 недели	– **сформировать предложения по архитектуре системы глубокого**	ПК-1, ПК-7, ПК-6,	вып.

Период практики	Содержание работ	Планируемые результаты практики (формируемые компетенции)	Отметка о выполнении *
	обучения (DL) для решения конкретной задачи – выбрать необходимые библиотеки для DL – **провести обучение** и **подобрать оптимальные параметры архитектуры DL** – **провести оценку метрик** качества DL	ПК-9, ПК-10	
4 неделя	– разработать прототип информационной системы / программного модуля для Front части ИС – сформировать материалы для проектной части выпускной квалификационной работы – Подготовить отчет по результатам прохождения производственной практики (преддипломной практики). Обсудить с руководителем выполненную работу. – Оформить отчет по практике. Выгрузить отчетную документацию в ЭИОС «Электронный университет».	ПК-1, ПК-6, ПК-7	вып.

Планируемые результаты практики

Коды компетенций	Содержание компетенции
ПК-1	Способен разрабатывать прикладное программное обеспечение, автоматизировать работу с базами данных и документами, программировать бизнес-логику приложений, выполнять интеграцию разнородных данных
ПК-6	Способен разрабатывать, настраивать и сопровождать информационные системы управления бизнесом
ПК-7	Способен использовать отечественные и международные стандарты при проектировании и обеспечении качества прикладного программного обеспечения
ПК-9	Способен разрабатывать методы извлечения, анализа и обработки информации
ПК-10	Способен применять математические методы моделирования процессов обработки информации с использованием средств интеллектуального анализа данных и машинного обучения

Планируемые результаты практики, соотнесенные с планируемыми результатами освоения образовательной программы

Код и наименование профессиональной компетенции выпускника программы бакалавриата	Код и наименование индикатора достижения профессиональной компетенции
ПК-1. Способен разрабатывать прикладное программное	ПК-1.1.

обеспечение, автоматизировать работу с базами данных и документами, программировать бизнес-логику приложений, выполнять интеграцию разнородных данных	Знает технологии программирования прикладного программного обеспечения и бизнес-логики приложений. ПК-1.2. Умеет разрабатывать и конфигурировать прикладное программное обеспечение. ПК-1.3. Владеет навыками автоматизации решения типовых задач, работы с базами данных и документами, интеграции разнородных данных в корпоративных информационных системах. ПК-1.4. Владеет навыками использования систем контроля версий
ПК-6. Способен разрабатывать, настраивать и сопровождать информационные системы управления бизнесом	ПК-6.1. Знает технологии совместной работы с документами, статистической обработки и распределенного хранения данных. ПК-6.2. Умеет разрабатывать информационные системы управления бизнесом и взаимоотношениями с клиентами. ПК-6.3. Владеет навыками сопровождения и настройки информационных систем управления бизнесом.
ПК-7. Способен использовать отечественные и международные стандарты при проектировании и обеспечении качества прикладного программного обеспечения	ПК-7.1. Знает правовые нормы, действующего законодательства, отечественные и международные стандарты в области информационных систем и технологий. ПК-7.2. Умеет использовать нормативно-правовые документы, международные и отечественные стандарты при решении стандартных проблем, возникающих в профессиональной деятельности. ПК-7.3. Владеет навыками в объеме, позволяющем использовать и составлять нормативно-правовые документы в своей профессиональной деятельности.
ПК-9. Способен разрабатывать методы извлечения, анализа и обработки информации	ПК-9.1. Знать теоретические и прикладные основы анализа больших данных. ПК-9.2. Уметь проводить анализ больших данных. ПК-9.3. Владеть методами извлечения информации и знаний из гетерогенных, мультиструктурированных и неструктурированных источников.
ПК-10. Способен применять математические методы моделирования процессов обработки информации с использованием средств интеллектуального анализа данных и машинного обучения	ПК-10.1. Знать принципы решения задач машинного обучения и интеллектуального анализа данных. ПК-10.2. Уметь создавать алгоритмические и математические модели прикладных задач интеллектуального анализа данных. ПК-10.3. Владеть навыками построения описательных и информационно-аналитических моделей для интеллектуального управления ресурсами предприятия.

Прим.: Перечень представленных заданий может быть изменен или дополнен руководителем практики от Университета.

Приложение к индивидуальному заданию

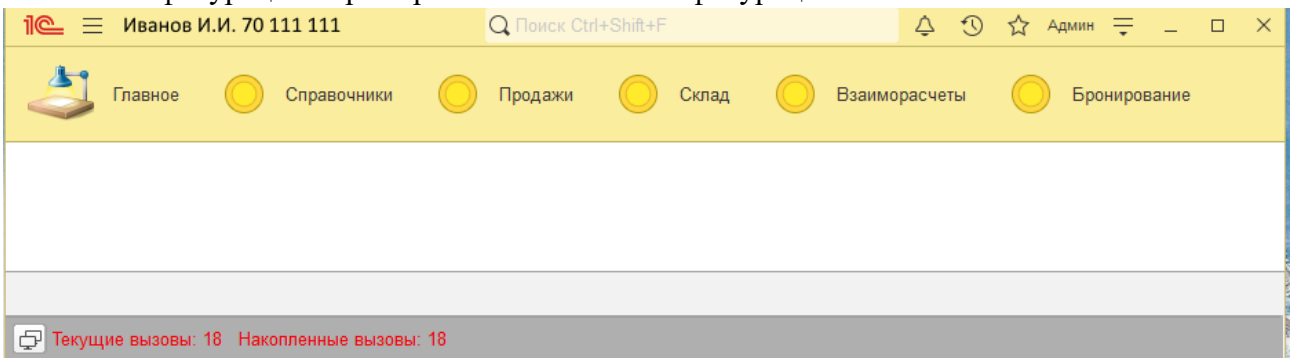
1. Требования к разрабатываемому программному обеспечению на базе платформы 1С:Предприятие

2.

1. Обязательна загрузка в ЭУ вместе с ВКР / выгрузка в репозиторий Gitflic или Github **файла с выгрузкой конфигурации 1С *.dt** (возможно приложить архив всей

папки с конфигурацией, с комментариями о версии платформы 1С, для которой выполнялся проект).

2. Рекомендуется при разработке конфигурации использовать 1С:EDT, к которой следует подключить Git-репозиторий в системе Gitflic или Github. Работа над ВКР должна представлять собой не менее 50 коммитов от имени студента, происходящих в течение периодов прохождения преддипломной практики и подготовки ВКР. Коммиты должны отражать итеративное развитие проекта, то есть не просто создание целиком готовых файлов, а их изменение по мере развития функционала проекта.
3. И файл конфигурации, и сама конфигурация должны называться по ФИО автора работы и его ID, также автор работы указан в качестве Поставщика в свойствах конфигурации. Пример наименования конфигурации:



4. Работа над проектом должна вестись в git-репозитории в системе Gitflic или Github. Работа над ВКР должна представлять собой не менее 50 коммитов от имени студента, происходящих в течение периодов прохождения преддипломной практики и подготовки ВКР. Коммиты должны отражать итеративное развитие проекта, то есть не просто создание целиком готовых файлов, а их изменение по мере развития функционала проекта.
5. Конфигурация должна содержать следующий минимальный набор объектов:
   - Справочники – **не менее 15**, из них **не менее 3 иерархических.**
   - Документы – **не менее 10**.
   - Константы – **не менее 2**.
   - Перечисления – **не менее 3**.
   - Общие модули – **не менее 1**.
   - Обработки – **не менее 1**.
   - Отчеты – **не менее 5**, в том числе сложные – **не менее 3**. Сложным отчетом считается отчет, для построения которого следует обратиться к данным **не менее 3-х объектов конфигурации**. Для решения задач учета следует использовать специализированные объекты (планы и регистры).
   - Подсистемы - **не менее 4.**
   - Регистры накопления – **не менее 3.**
6. Отчеты, представленные в конфигурации должны иметь **печатные формы**.
7. Проанализировать необходимость использования **регистров сведений** и **регистров расчета**. Разработать необходимые регистры сведений и регистры накопления. Анализ отразить в ВКР.
8. Разработать механизм проведения документов по регистрам накопления, а также по иным регистрам конфигурации, при наличии

9. Создать **не менее 3-х ролей пользователей** конфигурации **(уровней доступа)**, например, администратор, менеджер, клиент. При необходимости, указать пароли от учетных записей во Введении к ВКР
10. Объем собственноручно разработанного программного кода должен быть **не менее 500 логических строк кода**. Пустые строки, строки комментариев, строки только с управляющими символами подсчету не подлежат. Программный код, созданный автоматически с использованием платформы 1С:Предприятие не учитывается.
11. Для основных сценариев использования конфигурации должны быть созданы бизнес-процессы.
12. Заполнить конфигурацию тестовыми данными, соответствующими предметной области и выбранной организации. **Не менее 10 примеров для каждого объекта конфигурации.**

3. Требования к разрабатываемому десктопному ПО на языке высокого уровня

1. Программное обеспечение разрабатывается на языке **Python** с использованием сред разработки **PyCharm** / **GigaIDE** / **VS Code**.
2. Обязательна загрузка в ЭУ вместе с ВКР / выгрузка в репозиторий Gitflic или Github **исполняемого *.exe файла и файлов программного кода *.py** (при необходимости архивируется вся папка проекта, с комментариями об используемых дополнительных библиотеках и версиях).
3. Работа над проектом должна вестись в git-репозитории в системе Gitflic или Github. Работа над ВКР должна представлять собой не менее 50 коммитов от имени студента, происходящих в течение периодов прохождения преддипломной практики и подготовки ВКР. Коммиты должны отражать итеративное развитие проекта, то есть не просто создание целиком готовых файлов, а их изменение по мере развития функционала проекта.
4. Объем собственноручно разработанного программного кода должен быть **не менее 2 000 логических строк кода**. Пустые строки, строки комментариев, строки только с управляющими символами подсчету не подлежат.
5. В git-репозитории должен присутствовать в корневом каталоге файл **main.py**, который запускает приложение.
6. В git-репозитории должен присутствовать файл **requirements.txt**, описывающий список необходимых для работы приложения и модели библиотек, которые надо будет установить проверяющему в пустое виртуальное окружение. Файл необходимо положить в корневой каталог репозитория.
7. В ВКР должно быть приведено обоснование выбора всех используемых библиотек. **Следует использовать самые последние стабильные версии библиотек**. В работе не должны использоваться библиотеки, требующие для своей корректной работы покупки токенов, аккаунтов и т.п.
8. В программном коде должны быть подробные комментарии на русском языке.
9. Обязательно должен быть сформирован **исполняемый** файл **(*.exe и т.п.)**.
10. В качестве приоритетного решения следует рассматривать программное обеспечение, построенное с использованием клиент-серверной архитектуры. Разработка программного решения, не предполагающего сетевое взаимодействие должна быть обоснована.

11. Обеспечить хранение данных приложения в СУБД, поддерживающей SQL-запросы. Рекомендуется PostgreSQL. База данных должна содержать **не менее 10 таблиц** и должна быть представлена в репозитории вместе со всем проектом. Файлы базы данных должны быть размещены в git-репозитории. Файлы базы данных должны быть размещены в git-репозитории.
12. В приложении должно быть **не менее 10 оконных форм**.
13. Реализовать **не менее одного диалогового окна** для получения данных от пользователя со специальной формы.
14. Реализовать функционал доступа к файловой системе.
15. Реализовать панель управления / настройки в приложении, где добавить **не менее 5 пунктов меню**.
16. Реализовать возможность формирования документов в форматах **.docx, .xlsx** и т.п. из разрабатываемого приложения.
17. Предусмотреть **не менее 3-х ролей пользователей (уровней доступа)**, например, администратор, менеджер, клиент. Предоставить учетные данных всех созданных пользователей во Введении к ВКР.
18. Разработать панель администратора.
19. Разработать личный кабинет пользователя.
20. Предусмотреть функционал «Справка по системе», с указанием ФИО автора работы.

В случае разработки ПО **для микроконтроллеров и программируемых логических интегральных схем:**

21. Разработка кода должна выполняться с использованием аппаратного обеспечения.
22. Результатом выполнения выпускной квалификационной работы должен быть работающий программно-аппаратный комплекс.

4. Требования к разрабатываемым ботам для мессенджеров и социальных сетей на языке высокого уровня

1. Программное обеспечение разрабатывается на языке **Python** с использованием сред разработки **PyCharm** / **VS Code** / **GigaIDE** / **Google Colab (для мессенджеров и социальных сетей)** или **файлов веб-страниц в формате *.html, *.css, *.php, *.js**.
2. Обязательна загрузка в ЭУ вместе с ВКР / выгрузка в репозиторий Gitflic или Github **исполняемого *.exe файла и файлов программного кода *.py** (при необходимости архивируется вся папка проекта, с комментариями об используемых дополнительных библиотеках и версиях), либо **файла программного кода *.ipynb**, если работа выполнялась с использованием языка программирования Python и сервисов Google Colab.
3. Работа над проектом должна вестись в git-репозитории в системе Gitflic или Github. Работа над ВКР должна представлять собой не менее 50 коммитов от имени студента, происходящих в течение периодов прохождения преддипломной практики и подготовки ВКР. Коммиты должны отражать итеративное развитие проекта, то есть не просто создание целиком готовых файлов, а их изменение по мере развития функционала проекта.

4. Объем собственноручно разработанного программного кода должен быть **не менее 2 000 логических строк кода**. Пустые строки, строки комментариев, строки только с управляющими символами подсчету не подлежат.
5. В git-репозитории должен присутствовать в корневом каталоге файл **main.py**, который запускает приложение.
6. В git-репозитории должен присутствовать файл **requirements.txt**, описывающий список необходимых для работы приложения и модели библиотек, которые надо будет установить проверяющему в пустое виртуальное окружение. Файл необходимо положить в корневой каталог репозитория.
7. В ВКР должно быть приведено обоснование выбора всех используемых библиотек. **Следует использовать самые последние стабильные версии библиотек**. В работе не должны использоваться библиотеки, требующие для своей корректной работы покупки токенов, аккаунтов и т.п.
8. В программном коде должны быть подробные комментарии на русском языке.
9. Обязательно во Введении к ВКР должна быть представлена ссылка на бот для подключения к мессенджеру или социальной сети. Бот для сайта должен быть размещен на бесплатном хостинге.
10. Обеспечить хранение данных бота в СУБД, поддерживающей SQL-запросы. Рекомендуется PostgreSQL. База данных должна содержать **не менее 10 таблиц** и должна быть представлена в репозитории вместе со всем проектом. Файлы базы данных должны быть размещены в git-репозитории.
11. В боте должно быть **не менее 10 команд, из них не менее 5 с параметрами.**
12. Среди ответов бота должно быть **не менее 5 разновидностей** не текстовых вариантов ответа, содержащих **маркапы с кнопками, картинками и т.д.**
13. Реализовать **не менее одного сценария** для получения данных от пользователя.
14. Реализовать функционал доступа к файловой системе.
15. Реализовать функционал настройки бота, где добавить **не менее 5 пунктов меню**.
16. Реализовать возможность **логирования и выгрузки отчетов о действиях пользователей**.
17. Предусмотреть **не менее 3-х уровней доступа пользователей**, например, разработчик, администратор, пользователь. Предоставить учетные данных всех созданных пользователей во Введении к ВКР.
18. Разработать сценарий взаимодействия с ботом для администратора.
19. Разработать сценарий взаимодействия с ботом для рядового пользователя.
20. Предусмотреть функционал «/help», содержащий справку по боту и указание ФИО автора.

5.

6. Требования к разрабатываемому программному обеспечению для мобильных платформ

1. Приложение должно быть разработано под операционную систему Android. Язык написания приложения: Java или Kotlin.
2. Обязательна загрузка в ЭУ вместе с ВКР / выгрузка в репозиторий Gitflic или Github исполняемого ***.apk файла-приложения для Android** и исходных файлов с программным кодом.
3. В программном коде должны быть подробные комментарии на русском языке.
4. Работа над проектом должна вестись в git-репозитории в системе Gitflic или Github. Работа над ВКР должна представлять собой **не менее 50 коммитов от имени**

студента, происходящих **в течение периодов прохождения преддипломной практики и подготовки ВКР**. Коммиты должны отражать итеративное развитие проекта, то есть не просто создание целиком готовых файлов, а их изменение по мере развития функционала проекта.

5. **Использование готовых универсальных кроссплатформенных фреймворков запрещается**. Используемые средства разработки не регламентируются, однако рекомендуется использовать Android Studio для уменьшения проблем со сборкой приложения в аудитории.
6. Определить не менее **5 взаимосвязанных сущностей** в рамках предметной области.
7. Для организации сущностей использовать модель MVC или MVVM.
8. Для организации интерфейса в файлах представлений использовать не менее одного макета ConstraintLayout.
9. **Не менее одного макета** представления сформировать как для портретного, так и для альбомного режима.
10. Ресурсы, в том числе строковые, вынести из кода и расположить в отдельных предусмотренных для этого файлах и папках в структуре проекта.
11. Контроллеры представлений выполнять с помощью класса Fragment и библиотеки поддержки androidx. Разработать взаимодействие фрагментов на разных этапах жизненного цикла.
12. У каждой сущности должна быть форма редактирования объекта, а также форма списка объектов, построенная на базе класса RecyclerView.
13. Обеспечить хранение данных приложения в СУБД, поддерживающей SQL-запросы. База данных должна содержать **не менее 10 таблиц** и должна быть представлена в репозитории вместе со всем проектом. Файлы базы данных должны быть размещены в git-репозитории. Допустимо использование библиотек ORM для упрощения доступа к данным. Рекомендуется использование SQLite.
14. Реализовать панель управления в приложении, где добавить **не менее 5 пунктов меню.**
15. Реализовать **не менее одного диалогового окна** для получения данных от пользователя со специальной формы.
16. Добавить в функционал возможность вызова стандартных приложений через неявные намерения (Intent).
17. В основной сущности должен быть реализован функционал доступа к файловой системе и получение изображений с камеры.
18. Реализовать взаимодействие основной сущности с внешним веб-сервисом.
19. Предусмотреть функционал «Справка», с указанием ФИО автора работы.

7. Требования к разрабатываемому сетевому ресурсу (веб-сайта, порталу и т.п.)

1. Сетевой ресурс должен быть разработан с применением языков программирования – PHP, JavaScript или Python с использованием фреймворков Django / Flask.
2. Обязательна загрузка в ЭУ вместе с ВКР / выгрузка в репозиторий Gitflic или Github **файлов веб-страниц в формате *.html, *.css, *.php, *.js** (при необходимости архивируется вся папка проекта, с комментариями об используемых версиях) или **файлами программного кода *.py** (при необходимости архивируется вся папка проекта, с комментариями об используемых дополнительных библиотеках и версиях), если работа выполнялась с использованием языка программирования Python.

3. Работа над проектом должна вестись в git-репозитории в системе Gitflic или Github. Работа над ВКР должна представлять собой **не менее 50 коммитов от имени студента**, происходящих **в течение периодов прохождения преддипломной практики и подготовки ВКР**. Коммиты должны отражать итеративное развитие проекта, то есть не просто создание целиком готовых файлов, а их изменение по мере развития функционала проекта.
4. Сетевой ресурс обязательно должен быть размещен на бесплатном хостинге.
5. При работе над сетевым ресурсом **недопустимо подменять разработку** ресурса **разворачиванием готового программного обеспечения** и установку готовых дополнительных программных модулей для работы с ним. Допускается разработка собственных программных модулей для работы с имеющимися программными системами и/или модификация таковых модулей. **Запрещено использование CMS и готовых плагинов к ней**. Исключением для данного пункта могут быть работы, тема которых явно указывает на данный тип деятельности.
6. При разработке сетевых ресурсов студентом должно быть предложено программное и/или аппаратное решение, полностью работоспособное и готовое к внедрению в выбранный бизнес-процесс.
7. Объем программного кода должен быть **не менее 2 000 логических строк кода**, в том числе и строк, написанных с применением языка разметки. Пустые строки, строки комментариев, строки только с управляющими символами подсчету не подлежат.
8. Разработать структуру Web-страниц. Сетевой ресурс должен содержать **не менее 10 страниц**, **доступных для неавторизованного пользователя**.
9. Разработать макет каждой страницы сетевого ресурса.
10. Предусмотреть форму обратной связи.
11. Разработать **не менее 10 пунктов бокового и/или верхнего меню, «хлебные крошки».** «Хлебные крошки» должны быть реализованы на каждой странице сетевого ресурса.
12. Обеспечить хранение данных приложения в СУБД, поддерживающей SQL-запросы. База данных должна содержать **не менее 10 таблиц** и должна быть представлена в репозитории вместе со всем проектом. Файлы базы данных должны быть размещены в git-репозитории.
13. Реализовать функционал доступа к файловой системе.
14. Реализовать возможность формирования документов в форматах **.docx, .xlsx** и т.п. из разрабатываемого приложения.
15. Предусмотреть **не менее 3-х ролей пользователей (уровней доступа)**, например, администратор, менеджер, клиент. Предоставить учетные данных всех созданных пользователей во Введении к ВКР.
16. Разработать панель администратора сетевого ресурса. В панели администратора должно быть предусмотрено **не менее 5 страниц**, доступных только под данной учетной записью.
17. Разработать личные кабинеты пользователей сетевого ресурса. В личных кабинетах пользователей должно быть предусмотрено **не менее 5 страниц**, доступных только под данными учетными записями.
18. Обеспечить корректное отображение сетевого ресурса в основных браузерах (Mozilla Firefox, Chrome, Яндекс.Браузер, Microsoft Edge)
19. Обеспечить корректное отображение сетевого ресурса на разных устройствах (ПК, ноутбук, смартфон)

20. Текст статей в сетевом ресурсе должен быть читабельным, грамотно сформулированным, корректным и соответствовать предметной области ВКР. **Не должно быть пустых страниц в сетевом ресурсе.**
21. В подвале ресурса должно быть указание ФИО автора работы.

8.

9. Требования к разрабатываемому интеллектуальному сервису (с использованием машинного обучения, нейронных сетей и интеллектуального анализа данных)

1. Интеллектуальный сервис разрабатывается на языке Python с использованием сервисов Google Colab / Yandex DataSphere и т.п.
2. Обязательна загрузка в ЭУ вместе с ВКР **файла/ов программного кода *.ipynb** (при необходимости архивируется вся папка проекта, с комментариями об используемых дополнительных библиотеках и версиях, а также с используемыми датасетами). Обязательна выгрузка в репозиторий Gitflic или Github **файла/ов программного кода *.ipynb** (при необходимости архивируется вся папка проекта, с комментариями об используемых дополнительных библиотеках и версиях, а также с используемыми датасетами)
3. При использовании публичных датасетов (например, с ресурса kaggle) в коде должна быть предусмотрена возможность прямой загрузки данных по ссылке.
4. Работа над проектом должна вестись в git-репозитории в системе Gitflic или Github. Работа над ВКР должна представлять собой **не менее 50 коммитов от имени студента**, происходящих **в течение периодов прохождения преддипломной практики и подготовки ВКР**. Коммиты должны отражать итеративное развитие проекта, то есть не просто создание целиком готовых файлов, а их изменение по мере развития функционала проекта.
5. Объем собственноручно разработанного программного кода должен быть **не менее 2 000 логических строк кода**. Считаются только строчки алгоритмического кода, строки кода разметки типа HTML или JSON, пустые строки, строки комментариев, строки только с управляющими символами подсчету не подлежат.
6. Создаваемое приложение для демонстрации работы приложения обязательно должно позволять:
   - загружать и сохранять данные для обучения моделей из локальных файлов или из внешних хранилищ данных (типа kaggle или коннекторов для подключения хранилищ на облачных сервисах);
   - проводить обучение моделей, сохранять обученную модель в файл по результатам обучения;
   - загружать обученную модель из файла.
   - применять обученную или загруженную модель по назначению
   - оценивать и сохранять результаты работы нескольких вариантов моделей, если такое предусмотрено тематикой ВКР
7. Дизайн приложения должен быть отражён в ВКР. Загрузка данных в приложение отражается в ВКР в **разделе разработки ПО**, обучение моделей для приложения - в **разделе обучения моделей**, процесс применения модели и оценки её результатов – в **разделе тестирования модели**.
8. В git-репозитории должен присутствовать в корневом каталоге файл **main.py**, который запускает приложение для работы с моделью.

9. В git-репозитории должен присутствовать файл **requirements.txt**, описывающий список необходимых для работы приложения и модели библиотек, которые надо будет установить проверяющему в пустое виртуальное окружение. Файл необходимо положить в корневой каталог репозитория.
10. В ВКР должно быть приведено обоснование выбора всех используемых библиотек машинного обучения. **Следует использовать самые последние стабильные версии библиотек**. В работе не должны использоваться модели машинного обучения, требующие для своей корректной работы покупки токенов, аккаунтов и т.п.
11. Все демонстрируемые в работе модели должны быть сохранены в сдаваемом git-репозитории в файле с названием **model** и расширением, которое удобно для конкретной выбранной архитектуры (например, модели на PyTorch сохраняются с расширением **.pt**, то есть будет **model.pt**). Если в работе демонстрируется больше 1 модели, то к названию файла добавляется порядковый номер (напр., model1, model2 и т.д.). При проверке будет ожидаться, что работать эти модели будут так же, как описано в отчёте.
12. Для всех демонстрируемых моделей должны быть сохранены данные для их обучения. При проверке будет ожидаться, что модель, обученная на этих данных, будет совпадать по метрикам своей эффективности с моделью в файле и описанием этих метрик в отчёте.
13. При работе моделей, ожидается достижение уровня точности, сопоставимого с уровнем **Accuracy >= 70%**. Если к конкретной модели в конкретной предметной области этот уровень не применим, то с руководителем работы согласовывается другая метрика качества.
14. Продолжительность периода обучения модели (.fit) не должно превышать 6-ти часов при использовании ресурсов ускорителя уровня NVIDIA Tesla T4 с объемом видеопамяти 16Gb (или других ускорителей, сопоставимых по вычислительной мощности).
15. В программном коде должны быть подробные комментарии на русском языке.
16. Реализация GUI интеллектуального сервиса может быть в виде «оконных» или web форм, а также с использованием возможностей чат-ботов по отрисовке интерфейсных компонентов. Конкретный формат GUI согласовывается с руководителем ВКР.
17. В ВКР обязательно должно быть математическое описание выбранной модели.
18. В начале файла модели в блоке комментариев должна быть указана ФИО автора и тема ВКР.

Методологические требования к разработке интеллектуального сервиса

1. Основной формат демонстрации навыков – **создание, обучение, тестирование и улучшение модели машинного обучения** в выбранной предметной области, сопровождаемое **визуальным анализом** получаемых результатов. Формат сохраняется не зависимо от сложности модели.
2. Создание модели предполагает, что студент умеет применять основные способы создания моделей в программном коде, как в плане написания алгоритмов «с нуля», так и в плане использование стандартных готовых библиотек – от базовых библиотек типа NumPy и Pandas, до библиотек глубокого обучения типа PyTorch, Tensorflow и т.п.

3. Создание модели предполагает, что студент владеет не только типовыми способами создания моделей, но и способен адаптировать модель под работу в конкретной предметной области, осуществляя математически грамотный подбор гиперпараметров моделей, структуру ансамблевых моделей или топологий нейронных сетей.
4. Уверенное владение моделями моделей предполагает знакомство студентов с алгоритмами оптимизации и функциями подсчёта потерь, способность объяснить математический смысл этих алгоритмов и функций и их отношение к целевой функции, выражаемой моделью.
5. Обучение модели предполагает получение данных, на основе которых проводится обучение, подготовку данных к работе, а так же проведение самого процесса обучения модели с визуализацией промежуточных и итоговых результатов.
6. Получение данных предполагает, что студент знает откуда вообще можно брать данные для обучения моделей. Знаком как с публичными ресурсами типа kaggle, так и с подходами к самостоятельной добыче данных через web-скрейпинг, публичные API или через коннекторы для подключения хранилищ на облачных сервисах.
7. **Генерация случайных данных не допускается**, поскольку случайные данные автоматически лишают результаты работы конкретной практической ценности.
8. Получение данных предполагает, что студент понимает, какие форматы данных необходимы разным типам моделей для успешной работы в выбранной предметной области, способен привести исходные данные в нужный формат, произведя их очистку и необходимые трансформации, такие как нормализация, векторизация, приведение к тензорам, в том числе с использованием готовых конструкций загрузки данных в профессиональных библиотеках.
9. Получение данных предполагает способность студента определить, какие данные полезны, а какие нет, применяя для этого разные способы отбора и фильтрации признаков, в том числе на основе подходов визуального анализа.
10. Получение данных предполагает способность студента выделять из данных тренировочные и тестовые выборки, знать стандартные способы и инструменты такого выделения, уметь применять эти способы с учётом нужд предметной области и задачи анализа.
11. Обучение модели предполагает, что студент знает, как запустить такое обучение в рамках выбранного им способа создания модели. На минимальном уровне – умеет применять стандартные инструменты обучения моделей, представленных в рабочих библиотеках. На максимальном уровне – способен самостоятельно реализовать алгоритм обучения модели методом градиентного спуска.
12. Обучение многослойных моделей предполагает способность студента запускать автоматические фазы обучения (т.н. «эпохи»), отслеживать и визуализировать изменение эффективности модели на каждом этапе обучения.
13. По результатам обучения, в некоторых случаях - и в процессе обучения, необходимо проводить тестирование моделей. Для этого надо знать метрики оценки эффективности моделей, уметь производить тестирование и применять метрики к результатам тестирования, владеть инструментами визуализации метрик.
14. Тестирование моделей предполагает от студента способность не только произвести тестирование, но и сделать содержательные выводы из его результатов. Содержательными считаются выводы о работоспособности модели, полезности результатов обучения, может быть возможных способах улучшения модели.

15. Улучшение модели машинного обучение предполагает внесение изменений в модель с целью получить улучшенные результаты. От студента ожидается способность сформулировать предложение об улучшении модели по результатам первичного тестирования, обосновать его с помощью математического и визуального анализа, выполнить это улучшение (изменив параметры модели или пересоздав модель заново с новой топологией) и заново провести обучение и тестирование, оценив результаты улучшения (то есть фактически повторить весь путь с улучшенной моделью). Это должно быть отражено в ВКР.

Студенты должны демонстрировать все вышеперечисленные пункты по отношению к любому типу модели машинного обучения, представленному в стандартных рабочих библиотеках.

Таким образом, студент предоставляет не только работающую модель, но и приложение как среду, в которой он тестировал работу своей модели и в которой её работу может протестировать руководитель ВКР или другой проверяющий. **Отсутствие среды, в которой можно проверить работу модели, является нарушением правил сдачи работы**.

10.

11. Требования к разрабатываемому интерактивному графическому приложению (компьютерной игре)

12.

1. Интерактивное графическое приложение разрабатывается на языке **Python** с использованием интерпретатора **PyCharm** или **Google Colab**.
2. Обязательна загрузка в ЭУ вместе с ВКР **исполняемого *.exe файла** и выгрузка в репозиторий Gitflic или Github **файлов программного кода *.py** (при необходимости архивируется вся папка проекта, с комментариями об используемых дополнительных библиотеках и версиях) если работа выполнялась с использованием языка программирования Python, либо **файла программного кода *.ipynb**, если работа выполнялась с использованием языка программирования Python и сервисов Google Colab и Yandex DataSphere.
3. Работа над проектом должна вестись в git-репозитории в системе Gitflic или Github. Работа над ВКР должна представлять собой **не менее 50 коммитов от имени студента**, происходящих **в течение периодов прохождения преддипломной практики и подготовки ВКР**. Коммиты должны отражать итеративное развитие проекта, то есть не просто создание целиком готовых файлов, а их изменение по мере развития функционала проекта.
4. Объем собственноручно разработанного программного кода должен быть **не менее 2 000 логических строк кода**. Считаются только строчки алгоритмического кода, строки кода разметки типа HTML или JSON, пустые строки, строки комментариев, строки только с управляющими символами подсчету не подлежат.
5. В программном коде должны быть подробные комментарии на русском языке
6. При описании внутренней архитектуры приложения, студент должен описать **не менее 10 сущностей игровой системы** (из которых **минимум 5** должны быть реализованы самостоятельно) и **не менее 5 игровых систем** (из которых **минимум 3** должны быть реализованы самостоятельно).
7. При описании интерфейса приложения, студент должен описать **не менее 4 игровых экранов и не менее 15 элементов интерфейса** на этих экранах (из

которых **не менее 8** взяты не в готовым виде, а реализованы самостоятельно). Элементом интерфейса считается любой интерфейсный объект, реагирующий на события в игровой системе (например, шкала здоровья или индикатор возможности действия) или сам инициирующий событийную цепочку по факту ввода от игрока (например, кнопка).

8. Архитектура приложения обязательно должна содержать локально сохраняемые данные, такие как пользовательские настройки, сохранения, пароли и т.д.
9. Для реализации приложения можно использовать любые игровые движки и middleware, при этом имея ввиду требование о реализации достаточного объёма собственного функционала поверх заимствованного.
10. В сдаваемом git-репозитории должен в корневом каталоге присутствовать **файл readme.md**, описывающий способы управления игровым процессом, а также содержащий ссылки на скачивание запускаемых сборок игрового приложения.
11. Сборки проекта можно выгружать в git-репозиторий, или, что предпочтительнее, осуществить релиз своего приложения на специализированной площадке типа itch.io.
12. Необходимым условием является наличие сборки, которую можно запустить под Linux, для запуска на компьютерах Университета для проверки. **Сборки только под Windows недостаточно**. Если проект нельзя собрать для Linux или хотя бы для Web-запуска из браузера под Linux, то запускаемую сборку необходимо установить в соответствующий докер-контейнер.
13. В интерактивном графическом приложении должна быть предусмотрена возможность показа ФИО автора и темы ВКР.

Методологические требования к разработке интерактивного графического приложения

14. **Основной формат демонстрации навыков –** выполнение полного комплекса работ по разработке интерактивного графического приложения (далее «игрового приложения»), в частности: определение требований к игровому приложению, планирование отвечающего этим требованиям функционала, как в плане системной архитектуры, так и в плане графического интерфейса, с последующей реализацией этого функционала, его тестированием на предмет соответствия требованиям и улучшение приложения по результатам тестирования.
15. Формат демонстрации навыков в целом сохраняется не зависимо от природы конкретного игрового приложения. Планирование, реализация, тестирование и улучшения являются неотъемлемыми частями процесса создания любого такого приложения.
16. На этапе определения требований к игровому приложению возможна вариативность, в том числе и в плане демонстрируемых студентом навыков. Рассматривается три основных подхода к определению требований к игровому приложению: требования научной модели, требования дизайн-фокуса и требования коммерческого продукта. Эти три способа требуют демонстрирования разных наборов навыков на этапе определения требований к игровому приложению.
17. Определение требований к игровому приложению через требования научной модели используется когда целью работы ставится создание игровой модели какого-то сложного процесса. Примерами приложений, моделирующих подобные сложные процессы являются игровые приложения в стиле Microsoft Flight Simulator, Kerbal

Space Program, Civilisation. Требования к игровому приложению в этом случае проистекают из требований к точности реализации и визуализации работы научной модели и описанных в этой модели процессов.

18. При выборе этого способа определения требований к игровому приложению, наличие готовой научной модели, выраженной в соответствующем наборе научных публикаций, является необходимым граничным условием. Студент может взять реализацию игрового приложения на основе научной модели как задание только если предъявит руководителю научные публикации, описывающие соответствующую научную модель.
19. Если студент решает в своей работе определять требования к своему игровому приложению через призму требований научной модели, то от него будут ожидаться
    a. Знание о научных моделях в выбранной им предметной области, описывающих моделируемый процесс. Способность адресовать к научным публикациям, обосновывать ими элементы дизайна игрового приложения.
    b. Умение реализовывать выбранную научную модель в программном виде, как в плане внутренней рабочей логики этой модели, так и в плане визуализации её работы и демонстрации возникающей в этой модели интересных процессов.
20. Определение требований к игровому приложению на основании дизайн-фокуса предполагает наличие внешнего бизнес-процесса или группы бизнес-процессов, в котором существует проблема. Решение этой проблемы и становится дизайн-фокусом игрового приложения. Требования к игровому приложению в этом случае определяются на основании бизнес-анализа исходной группы бизнес-процессов и определения точек применения игрового приложения для разрешения проблем в этих процессах.
21. При выборе этого способа определения требований к игровому приложению, необходимо наличие у студента доступа к специалисту, который поможет ему описать конкретные бизнес-процессы и создаст тем самым возможность для их анализа с целью последующего выделения дизайн-фокуса игрового приложения. Студент может использовать в своей работе бизнес-процессы Университета (и тогда специалистом может выступать руководитель работы) или внешней организации (и тогда специалист должен быть из внешней организации). Отсутствие доступа к специалисту делает невозможным определение требований к игровому приложению через дизайн-фокус.
22. Если студент решает в своей работе определять требования к своему игровому приложению через дизайн-фокус, то от него будут ожидаться:
    a. Навыки анализа бизнес-процессов и выделения в них тех точек, которые могут быть улучшены с помощью игрового приложения.
    b. Знание технологий и подходов геймификации бизнес-процессов и способность применить эти знания для предложения улучшения в бизнес-процессах за счёт игрового приложения.
23. Определение требований к игровому приложению на основании требований коммерческого продукта предполагает, что студент в состоянии определить провести маркетинговое профилирование целевой аудитории продукта, может сформулировать потребности этой целевой аудитории и тем самым обосновать требования к игровому приложению как потребительским характеристик, интересующих целевую аудиторию.

24. Если студент решает в своей работе определять требования к своему игровому приложению как к коммерческому продукту, то от него будут ожидаться:
    a. Навыки поиска других продуктов в выбранной им категории игровых приложений, определения тех их ключевых качеств, на которые реагирует аудитория.
    b. Способность профилирования аудитории выбранной категории игровых приложений с точки зрения интересующего конкретных представителей конкретного профиля набора потребительских качеств.
    c. Способность выделить целевую аудиторию для своего продукта и спланировать потребительские качества собственного продукта, соответствующие маркетинговому профилю целевой аудитории.
25. Сформулировав требования к своему игровому приложению одним из трёх вышеописанных способов, студент переходит к планированию реализации игрового приложения. Необходимо спланировать первую версию приложения, удовлетворяющего заданным требованиям, как в плане программной архитектуры, так и в плане интерфейсов и игровых взаимодействий.
26. На этапе планирования от студента ожидаются:
    a. Знания о типовых способах составления проектной документации для игрового приложения (далее «дизайн-документа»).
    b. Умение правильно структурировать дизайн-документ, отразив в нём как требования к игровому продукту, так и способы их реализации.
    c. Владение способами визуализации схем игровых взаимодействий и необходимых алгоритмов, видов игровых объектов и отношений между ними, способов управления игровым процессом.
27. Спланировав первую версию приложения, студент переходит к его реализации. На этом этапе, от студента ожидаются:
    a. Знания общих архитектурных схем, стандартных алгоритмов и типовых структур данных, позволяющих реализовать запланированное в дизайн-документе.
    b. Умение преобразовывать требования продукта (будь то потребительские качества, требования научной модели или схемы геймификации) в конкретный функционал игрового приложения.
    c. Владение инструментарием создания игровых приложений, будь то промежуточные библиотеки типа pygame или Ren’Py, или интегрированные решения типа Unity.
28. Реализовав первую версию приложения, студент должен протестировать её относительно требований и сделать выводы о необходимых улучшениях. Конкретный характер тестов будет зависеть от способа формирование требований.
    a. В случае научной модели, необходимо реализовать архитектуру автоматического тестирования корректного выполнения научной модели во всех игровых сценариях, показать работу этой архитектуры.
    b. В случае коммерческого продукта, необходимо предложить игровой продукт целевой аудитории, собрать с неё обратную связь. После чего на основании этой обратной связи сделать выводы о том, насколько продукт отвечает потребностям аудитории в рамках сформулированного профиля, предложить и реализовать улучшения продукта.
    c. В случае дизайн-фокуса, необходимо согласовать конкретную схему проверки со специалистом, владеющим улучшаемым с помощью игрового

приложения бизнес-процессом. Согласованную схему проверки будет необходимо выполнить, отразить в отчёте работы и после чего, на основании результатов, сделать выводы о работоспособности предложенной геймификации и внести улучшения.

В итоге, от студентов ожидается внятное объяснение требований, которые они сформулировали к реализуемому игровому приложению, способов выполнения этих требований, способов верификации выполнения этих требований, а также улучшений, которые были сделаны в тех местах, где требования с первого раза не выполнялись.

13. Требования к тестированию программного продукта

1. Для разрабатываемого приложения необходимо выбрать **минимум 4 метода тестирования** из нижеприведенных видов тестирования:

№	Вид тестирования	Метод тестирования	Описание метода
1	По принципам работы с приложением	Позитивное тестирование	тестирование, при котором используются только корректные данные.
		Негативное тестирование	тестирование приложения, при котором используются некорректные данные и выполняются некорректные операции
2	По уровню функциоанльного тестирования	Смоук-тестирование (дымовое, **smoke test**)	тестирование, выполняемое на новой сборке, с целью подтверждения того, что программное обеспечение стартует и выполняет основные для бизнеса функции.
		Тестирование критического пути (**critical path)**	направлено для проверки функциональности, используемой обычными пользователями во время их повседневной деятельности.
		Расширенное тестирование (**extended)**	направлено на исследование всей заявленной в требованиях функциональности.
3	В зависимости от исполнителей	Альфа-тестирвоание	является ранней версией программного продукта. Может выполняться внутри организации-разработчика с возможным частичным привлечением конечных пользователей.
		Бета-тестирование	программное обеспечение, выпускаемое для ограниченного количества пользователей. Главная цель — получить отзывы клиентов о продукте и внести соответствующие изменения.
4	В зависимости от целей тестирования	Функциональное тестирование **(functional testing)**	направлено на проверку корректности работы функциональности приложения.

		Нефункциональное тестирование**(non-functional testing)**:	тестирование атрибутов компонента или системы, не относящихся к функциональности.
		– Тестирование производительности **(performance testing)**	определение стабильности и потребления ресурсов в условиях различных сценариев использования и нагрузок.
		– Нагрузочное тестирование **(load testing)**	определение или сбор показателей производительности и времени отклика программно-технической системы или устройства в ответ на внешний запрос с целью установления соответствия требованиям, предъявляемым к данной системе (устройству).
		– Тестирование масштабируемости **(scalability testing)**	тестирование, которое измеряет производительность сети или системы, когда количество пользовательских запросов увеличивается или уменьшается.
		– Объемное тестирование **(volume testing)**	это тип тестирования программного обеспечения, которое проводится для тестирования программного приложения с определенным объемом данных.
		– Стрессовое тестирование **(stress testing)**	тип тестирования направленный для проверки, как система обращается с нарастающей нагрузкой (количеством одновременных пользователей).
		– Инсталляционное тестирование **(installation testing)**	тестирование, направленное на проверку успешной установки и настройки, обновления или удаления приложения.
		– Тестирование интерфейса **(GUI/UI testing)**	проверка требований к пользовательскому интерфейсу.
		– Тестирование удобства использования **(usability testing)**	это метод тестирования, направленный на установление степени удобства использования, понятности и привлекательности для пользователей разрабатываемого продукта в контексте заданных условий.
		– Тестирование локализации **(localization testing)**	проверка адаптации программного обеспечения для определенной аудитории в соответствии с ее культурными особенностями.
		– Тестирование безопасности **(security testing)**	это стратегия тестирования, используемая для проверки безопасности системы, а также для анализа рисков, связанных с обеспечением целостного подхода к защите приложения, атак хакеров,

			вирусов, несанкционированного доступа к конфиденциальным данным.
		– Тестирование надежности **(reliability testing)**	один из видов нефункционального тестирования ПО, целью которого является проверка работоспособности приложения при длительном тестировании с ожидаемым уровнем нагрузки.
		– Регрессионное тестирование **(regression testing)**	тестирование уже проверенной ранее функциональности после внесения изменений в код приложения, для уверенности в том, что эти изменения не внесли ошибки в областях, которые не подверглись изменениям.
		– Повторное/**подтверждающее** тестирование **(re-testing/confirmation testing)**	тестирование, во время которого исполняются тестовые сценарии, выявившие ошибки во время последнего запуска, для подтверждения успешности исправления этих ошибок.
5	По запуску кода на исполнение	Статическое	процесс тестирования, который проводится для верификации практически любого артефакта разработки: программного кода компонент, требований, системных спецификаций, функциональных спецификаций, документов проектирования и архитектуры программных систем и их компонентов.
		Динамическое	тестирование проводится на работающей системе, не может быть осуществлено без запуска программного кода приложения.
6	По доступу к коду и архитектуре	Метод белого ящика	метод тестирования ПО, который предполагает полный доступ к коду проекта.
		Сетод серого ящика	метод тестирования ПО, который предполагает частичный доступ к коду проекта (комбинация White Box и Black Box методов).
		Метод черного ящика	метод тестирования ПО, который не предполагает доступа (полного или частичного) к системе. Основывается на работе исключительно с внешним интерфейсом тестируемой системы.
7	По уровню детализации приложения	Модульное тестирование	проводится для тестирования какого-либо одного логически выделенного и изолированного элемента (модуля) системы в коде. Проводится самими разработчиками, так как предполагает полный доступ к коду.

		Интеграционное тестирование	тестирование, направленное на проверку корректности взаимодействия нескольких модулей, объединенных в единое целое.
		Системное тестирование	процесс тестирования системы, на котором проводится не только функциональное тестирование, но и оценка характеристик качества системы — ее устойчивости, надежности, безопасности и производительности.
		Приемочное тестирование	проверяет соответствие системы потребностям, требованиям и бизнес-процессам пользователя.
8	По степени автоматизации	Ручное тестирование	
		Автоматическое тестирование	

2. **Сформировать тест план.** Тест план должен отвечать на следующие вопросы: Что необходимо протестировать? Как будет проводиться тестирование? Когда будет проводиться тестирование? Критерии начала тестирования. Критерии окончания тестирования.

   Основные пункты тест плана:
   - Идентификатор тест плана (Test plan identifier);
   - Введение (Introduction);
   - Объект тестирования (Test items);
   - Функции, которые будут протестированы (Features to be tested;)
   - Функции, которые не будут протестированы (Features not to be tested);
   - Тестовые подходы (Approach);
   - Критерии прохождения тестирования (Item pass/fail criteria);
   - Критерии приостановления и возобновления тестирования (Suspension criteria and resumption requirements);
   - Результаты тестирования (Test deliverables);
   - Задачи тестирования (Testing tasks);
   - Ресурсы системы (Environmental needs);
   - Обязанности (Responsibilities);
   - Роли и ответственность (Staffing and training needs);
   - Расписание (Schedule);
   - Оценка рисков (Risks and contingencies);
   - Согласования (Approvals).
3. **Сформировать тест-кейсы** (тестовые сценарии). Обязательные атрибуты тест кейса:
   - **Предусловия** (PreConditions) — список действий, которые приводят систему к состоянию пригодному для проведения основной проверки. Либо список условий, выполнение которых говорит о том, что система находится в пригодном для проведения основного теста состояния.
   - **Шаги** (Steps) — список действий, переводящих систему из одного состояния в другое, для получения результата, на основании которого можно сделать вывод о удовлетворении реализации, поставленным требованиям.

- **Ожидаемый результат** (Expected result) — что по факту должны получить.

4. На все найденные ошибки **сформировать баг-репорт**. Баг-репорт должен обязательно содержать атрибуты: приоритет и серьезность.
   - **Серьёзность** показывает, насколько баг влияет на возможность работать в программе. Виды:

Серьёзность бага	**Как влияет на систему**
S1 — Блокирующая (Blocker).	Приложение не запускается ни на одном устройстве или выдаёт ошибку при загрузке. *Пример: приложение не открывается, поэтому невозможно сделать покупку.*
S2 — Критическая (Critical).	Часть функционала не работает, но это не блокирует процесс. *Пример: Скидка по купону не работает, но товар всё равно можно купить без скидки.*
S3 — Значительная (Major).	Часть логики работает некорректно, но пользователь может решить проблему другим способом. *Пример: поиск в приложении не выдаёт нужный товар, но его можно найти, если искать по брендам.*
S4 — Незначительная (Minor).	Не нарушает логику приложения. *Пример: кнопка «Купить» уехала за экран и видна только наполовину.*
S5 — Тривиальная (Trivial).	Не относится к логике приложения и не влияет на общее качество продукта. *Пример: малозаметная опечатка в меню.*

   Приоритет — это критерий, который показывает, насколько быстро нужно исправить дефект. Приоритет может быть:
   - высокий — исправить в первую очередь;
   - средний — исправить, когда разобрались с первой категорией багов;
   - низкий — исправить, когда разобрались с багами других приоритетов.

5. Произвести отладку разработанной системы.